

Experiment 7: Cross-Site Scripting (XSS) Testing

Aim

To manually test a web application's input fields for Cross-Site Scripting (XSS) vulnerabilities by injecting sample scripts and observing whether they execute.

Requirements

A lab machine (Kali Linux recommended) with the required tools installed and an authorized target machine on the same isolated lab network (e.g., Metasploitable2 / DVWA).

Algorithm / Procedure

1. Open the target web application (e.g., DVWA XSS module) in a browser.
2. Identify input fields that reflect or store user input (search box, comment box, name field, etc.).
3. Enter a simple, harmless test payload into the field.
4. Submit the form and observe whether an alert box pops up (this confirms the input was not sanitized).
5. Repeat with a stored-XSS test in a field that saves data (e.g., a guestbook/comment) to check if the payload executes for other users too.
6. Record which fields are vulnerable (reflected vs stored XSS).
7. Document the fix: proper output encoding/escaping and input validation on the server side.

Sample Command / Code

```
# Basic reflected XSS test payload
<script>alert('XSS Test')</script>
```

```
# Payload to test if it fires via an image tag (in case <script> is filtered)
<img src=x onerror=alert('XSS')>
```

```
# Payload to test a stored XSS in a comment field
<script>alert(document.cookie)</script>
```

Note: These are the standard, well-known 'alert()' test payloads used purely to confirm whether input is sanitized – they only pop up a harmless message box in your own test browser and cause no real effect.

Result

The above procedure was successfully executed in the lab environment and the observed output was recorded/verified. (Attach screenshot of your terminal output here.)

Output:-

testfire.net says
XSS TEST

OK

~~OK~~